

Lab 12 - Privacy-Preserving Data Encryption

This week you will be working with two forms of privacy-preserving encryption: Multi-Party Computation, specifically Private Set Intersection, and Partially Homomorphic.

Multi-Party Computation allows multiple people or entities to collaboratively compute a value using their private data without revealing any of those inputs to each other. As you can imagine, this is valuable when the data collected and being analyzed needs privacy and confidentiality protections. Examples would be in financial transactions, health care information exchange, or any personally identifiable information.

Fully Homomorphic Encryption allows computation to be performed directly on encrypted data. The data can remain confidential while being processed. One groundbreaking possibility for this type of encryption is the ability for secure cloud computing without exposing sensitive data. If it is a fully homomorphic implementation, any arbitrary computation on ciphertexts may be undertaken. One possible library to use in python for FHE would be `openfhe`. However, we are going to work with Partially Homomorphic Encryption (PHE) since we do not need arbitrary computations or long paths (and it is easier to work with)!

Part 01- Private Set Intersection

A specific application of Multi-Party Computation (MPC) is Private Set Intersection (PSI). This allows two or more parties to find the intersections of their databases while the actual data in the dataset remains private. The example in lecture was where Bob and Alice wanted to find the words they shared in their private word lists. A real world example would be where two companies or their representatives wanted to find common clients without revealing their entire client database. Only those intersecting elements are revealed to both parties.

Informational: During the COVID-19 pandemic PSI was used for contact tracing efforts that allowed individual privacy. This was a way that individuals could track their own movement and interpersonal interactions without providing the details to a third party. The general way PSI was used during COVID-19 pandemic was

1. Individuals' devices collected proximity data, many times through Bluetooth. This data was tokenized.
2. The tokens were stored locally ensuring their contacts with others were not shared with a central authority.

3. When a user tested positive for COVID-19 they could voluntarily share their tokens with the health authority. PSI was used to compare the infected individual's tokens with other users who chose to upload or check their tokens against the reported infected tokens.
4. Only the intersection (that a match existed, not who the match was) was disclosed. This notification allowed the potentially exposed individuals to take appropriate actions such as self-isolation or testing without the identity of the infected person being given.

Lab Starts Here: For COVID-19, oblivious programmable pseudo-random functions (OPPRFs) were used. Many times the function chosen for the COVID-19 implementation was a cryptographic hash. While we don't implement a full blown execution of uploading movement tokens and contact tracing, we will have some location and timestamp data which has been hashed to represent the individual who has acquired COVID-19. You will compare the hashes of "your movements" on three days that have been provided to you to see if you were in contact with the infected individual.

1. Log into your Ubuntu box in iselab. You will need to scp the skeleton code and data files from the repo server. Make sure you are in your homework directory.
 - a. `scp -r repo@repo.homework.3310.com:/home/repo/lab12 .`
2. In the part01 folder edit the `part01_patient_tracing_skel.py`. The code has been provided for you.
 - a. There is no TODO, even though the skeleton code indicates there is. I wrote it for you this semester.
 - b. The program reads in the json file containing the hashes of the movements of the person (patient) who contracted COVID-19.
 - i. As you know, json is a file written with key:value pairs. If you open the `hashedPatientData.json` file in a text editor you will notice the keys are the hashes of the location/time combination. The value is a string "present". I chose to use json to store the hashes because there is an easy way for you to find the intersection of keys. However, that meant I had to supply something as a placeholder. I chose the string "present." I could have used "pbj" since it has no impact on what we are trying to do.
 - c. You have been provided with your locations in a file called `"studentLocations.json"`. The program is set to run for all dates in the file.
 - i. `python3 part01_patient_tracing_skel.py`
 - d. The program hashes those location/time combinations for you.
 - e. The program will find the intersection of those location/time combinations. *In the real world you would never share that entry data in plaintext.*
 - f. **On which days were you in contact with the infected individual? Take a screenshot of the location and time where you were in contact with the COVID-19 patient to include in your lab report.**
 - g. **What do you know about the infected person?**
 - h. **Identify a flaw with the use of hashing in this scenario.** Consider how errors and inaccuracies in time and location might impact the hash.

- i. **Please provide another real-world example where PSI could be beneficial. Justify your reasons why it maintains confidentiality and privacy in your proposed use case. Explain how it benefits public good and protects individual rights.**

Part 02 - Partially Homomorphic Encryption - Calculate

3. Install the Partially Homomorphic Encryption package in python (phe) so you can implement the Paillier encryption scheme. Paillier is known for its ability to conduct addition and multiplication without needing to decrypt the values first. You have to run this in a virtual environment because the package is not maintained by Debian.
 - a. `sudo apt install python3.12-venv`
 - b. `python3 -m venv phe_env`
 - c. `source phe_env/bin/activate`
 - d. `pip install phe`
4. In this scenario, you are the researcher for a drug company who has a new blood pressure medicine in trials at four hospitals in the Midwest. You have gathered four data points in the study: systolic blood pressure (SBP), diastolic blood pressure (DBP), heart rate (HR), and a patient satisfaction score reflecting the patients' experiences with medicine and their overall treatment. Upon gathering the data from the four hospitals, you now need to analyze the data. However, you want to use the processing power of the cloud, but can't share the actual data points. You decided to use Partially Homomorphic Encryption (phe) to encrypt the data so you can securely analyze the data in the cloud.
5. You will need to edit `part02_drug_trial_cloud_calcs_skel.py`. You will need to sum the encrypted values for each measurement provided by the four hospitals. In other words, sum the 4 values in index 0 which is SBP, the values in index 1 which is DBP, etc. You treat the addition just like any other addition. The oddity is that while you can print the value, it will have no real meaning.
6. Once you have the 4 measurements summed, you will calculate the average over the 4 hospitals. Again, the mathematical function is simple. The only real way to check that you are successful is to print the values to see if something is actually being calculated and then checking in part03 if your averages make any sense once you decrypt them.
7. **Upload your code to Canvas.**

Part 03 - Partially Homomorphic Encryption - Decrypt

8. Now that you “used the cloud” to calculate your encrypted averages, you can “download” those averages (not really it is all on your Ubuntu desktop) and then decrypt them. You will use `part03_drug_trial_research_skel.py` to do this. Again, most of the code is implemented for you.
9. You have the private key in your control because you generated the public/private key pair when you encrypted the original data. You will use the private key to decrypt each of the averages. You should print these to the screen and **take a screenshot of the result to include in your lab report.**
10. **Upload your code to Canvas.**
11. Consider a voting application for a state or national election. **How could partially homomorphic encryption be effectively used?** Give a detailed example from three viewpoints. The most basic question is should they be able to see the plaintext vote for the candidate. You can expand from there.
 - a. From the poll workers’ vantage point. The poll worker is the person who manages the polling place by setting up the equipment, check-in voters, and ensures the election laws are followed.
 - b. From the vote counters’ vantage point. The vote counters tally the votes and ensure accuracy including audits and recounts.
 - c. From the election officials’ perspective. The election officials compile the results from each polling station and report them to the public and the relevant authorities. They also certify the election results.

Lab 12 Template

Some questions require multiple parts to be answered, be sure to discuss them in full.

Part 01:

1. **On which days** were you in contact with the infected individual? **Take a screenshot** of the location and time where you were in contact with the COVID-19 patient to include in your lab report.
(10 points)
2. **What do you know about the infected person?**
(5 points)
3. **Identify a flaw with the use of hashing in this scenario.** Consider how errors and inaccuracies in time and location might impact the hash.
(10 points)

4. Please provide **another real-world example where PSI could be beneficial**. Justify your reasons why it maintains **confidentiality** and **privacy** in your proposed use case. Explain how it benefits **public good** and protects **individual rights**.
(25 total points, 5 each set of bold words)

Part 02:

5. **Upload** your code to Canvas.
(10 points)

Part 03:

6. **Take a screenshot of the 4 plaintext averages**.
(10 points)
7. **Upload** your code to Canvas.
(5 points)
8. **How could partially homomorphic encryption be effectively used in an election?** 3 viewpoints
(25 total points, 8.33 points each perspective)